MongoDB Operators & Queries

Fake Data তৈরি

🌍 Real World Example

ধরুন আপনি একটা Job Portal তৈরি করছেন — যেমন Bdjobs বা LinkedIn। সেখানে লক্ষাধিক চাকরির তথ্য থাকবে। কোন কোম্পানিতে কত বেতন? কোন স্কিল দরকার? কতজন Apply করেছে? এই সব তথ্য MongoDB-তে রাখা হয়। আমরা ঠিক এইরকম একটা Job Collection তৈরি করবো যেখানে ২০টি ফেক জব ডেটা থাকবে এবং এই পুরো মডিউলে আমরা এই ডেটার উপরেই সব কুয়েরি প্র্যাকটিস করবো।

📦 Fake Job Data ইনসার্ট করুন

এই ২০টি ফেক জব ডেটাই পুরো মডিউলে ব্যবহার হবে:

```
[
  {
    "title": "Software Engineer",
    "company": "TechCorp Bangladesh",
    "location": "Dhaka",
```

```
  "salary": 75000,
  "experience": 3,
  "skills": ["JavaScript", "Node.js", "MongoDB"],
  "isRemote": false,
  "applicants": 120,
  "posted": "2024-01-15T00:00:00.000Z",
  "status": "active",
  "department": "Engineering"
},
{
  "title": "Frontend Developer",
  "company": "Digital Solutions",
  "location": "Chittagong",
  "salary": 55000,
  "experience": 2,
  "skills": ["React", "CSS", "JavaScript"],
  "isRemote": true,
  "applicants": 85,
  "posted": "2024-02-10T00:00:00.000Z",
  "status": "active",
  "department": "Engineering"
},
{
  "title": "Data Analyst",
  "company": "InsightHub",
  "location": "Dhaka",
  "salary": 60000,
  "experience": 4,
  "skills": ["Python", "SQL", "Excel"],
```

```
  "isRemote": false,
  "applicants": 200,
  "posted": "2024-01-20T00:00:00.000Z",
  "status": "active",
  "department": "Analytics"
},
{
  "title": "DevOps Engineer",
  "company": "CloudBase Ltd",
  "location": "Sylhet",
  "salary": 90000,
  "experience": 5,
  "skills": ["Docker", "Kubernetes", "AWS"],
  "isRemote": true,
  "applicants": 45,
  "posted": "2024-03-01T00:00:00.000Z",
  "status": "closed",
  "department": "Infrastructure"
},
{
  "title": "UI/UX Designer",
  "company": "CreativeMinds",
  "location": "Dhaka",
  "salary": 50000,
  "experience": 2,
  "skills": ["Figma", "Adobe XD", "Sketch"],
  "isRemote": false,
  "applicants": 150,
  "posted": "2024-02-25T00:00:00.000Z",
```

```
  "status": "active",
  "department": "Design"
 },
 {
  "title": "Backend Developer",
  "company": "ServerStack",
  "location": "Dhaka",
  "salary": 80000,
  "experience": 4,
  "skills": ["Python", "Django", "PostgreSQL"],
  "isRemote": true,
  "applicants": 95,
  "posted": "2024-01-05T00:00:00.000Z",
  "status": "active",
  "department": "Engineering"
 },
 {
  "title": "Project Manager",
  "company": "ManagePro BD",
  "location": "Dhaka",
  "salary": 95000,
  "experience": 7,
  "skills": ["Agile", "Scrum", "Jira"],
  "isRemote": false,
  "applicants": 60,
  "posted": "2024-03-10T00:00:00.000Z",
  "status": "active",
  "department": "Management"
 },
```

```
{
  "title": "QA Engineer",
  "company": "BugFree Tech",
  "location": "Rajshahi",
  "salary": 45000,
  "experience": 1,
  "skills": ["Selenium", "JIRA", "Testing"],
  "isRemote": false,
  "applicants": 30,
  "posted": "2024-02-15T00:00:00.000Z",
  "status": "active",
  "department": "Quality"
},
{
  "title": "Machine Learning Engineer",
  "company": "AIVentures",
  "location": "Dhaka",
  "salary": 120000,
  "experience": 6,
  "skills": ["Python", "TensorFlow", "ML"],
  "isRemote": true,
  "applicants": 75,
  "posted": "2024-03-05T00:00:00.000Z",
  "status": "active",
  "department": "AI"
},
{
  "title": "Database Administrator",
  "company": "DataKeep BD",
```

```
  "location": "Dhaka",
  "salary": 70000,
  "experience": 5,
  "skills": ["MySQL", "MongoDB", "Redis"],
  "isRemote": false,
  "applicants": 40,
  "posted": "2024-01-30T00:00:00.000Z",
  "status": "closed",
  "department": "Engineering"
},
{
  "title": "Mobile Developer",
  "company": "AppFactory",
  "location": "Chittagong",
  "salary": 65000,
  "experience": 3,
  "skills": ["Flutter", "React Native", "Dart"],
  "isRemote": true,
  "applicants": 110,
  "posted": "2024-02-20T00:00:00.000Z",
  "status": "active",
  "department": "Mobile"
},
{
  "title": "Cybersecurity Analyst",
  "company": "SecureNet BD",
  "location": "Dhaka",
  "salary": 85000,
  "experience": 4,
```

```
  "skills": ["Ethical Hacking", "Firewalls", "SIEM"],
  "isRemote": false,
  "applicants": 25,
  "posted": "2024-03-08T00:00:00.000Z",
  "status": "active",
  "department": "Security"
},
{
  "title": "Technical Writer",
  "company": "DocsFirst",
  "location": "Remote",
  "salary": 40000,
  "experience": 2,
  "skills": ["Markdown", "Technical Writing", "Git"],
  "isRemote": true,
  "applicants": 55,
  "posted": "2024-02-05T00:00:00.000Z",
  "status": "active",
  "department": "Documentation"
},
{
  "title": "Cloud Architect",
  "company": "SkyInfra",
  "location": "Dhaka",
  "salary": 130000,
  "experience": 8,
  "skills": ["AWS", "Azure", "GCP"],
  "isRemote": true,
  "applicants": 35,
```

```
  "posted": "2024-01-25T00:00:00.000Z",
  "status": "active",
  "department": "Infrastructure"
 },
 {
  "title": "React Developer",
  "company": "WebWorks BD",
  "location": "Dhaka",
  "salary": 58000,
  "experience": 2,
  "skills": ["React", "Redux", "TypeScript"],
  "isRemote": false,
  "applicants": 180,
  "posted": "2024-03-12T00:00:00.000Z",
  "status": "active",
  "department": "Engineering"
 },
 {
  "title": "HR Manager",
  "company": "PeopleFirst BD",
  "location": "Dhaka",
  "salary": 55000,
  "experience": 5,
  "skills": ["Recruitment", "HR Policies", "Payroll"],
  "isRemote": false,
  "applicants": 70,
  "posted": "2024-02-28T00:00:00.000Z",
  "status": "active",
  "department": "HR"
```

```
},
{
  "title": "Network Engineer",
  "company": "NetConnect BD",
  "location": "Khulna",
  "salary": 62000,
  "experience": 3,
  "skills": ["Cisco", "Networking", "Firewall"],
  "isRemote": false,
  "applicants": 20,
  "posted": "2024-01-18T00:00:00.000Z",
  "status": "closed",
  "department": "Infrastructure"
},
{
  "title": "Blockchain Developer",
  "company": "ChainTech BD",
  "location": "Dhaka",
  "salary": 110000,
  "experience": 5,
  "skills": ["Solidity", "Web3.js", "Ethereum"],
  "isRemote": true,
  "applicants": 15,
  "posted": "2024-03-15T00:00:00.000Z",
  "status": "active",
  "department": "Blockchain"
},
{
  "title": "Business Analyst",
```

```
  "company": "Analyse Pro",
  "location": "Dhaka",
  "salary": 68000,
  "experience": 4,
  "skills": ["Excel", "PowerBI", "SQL"],
  "isRemote": false,
  "applicants": 90,
  "posted": "2024-02-12T00:00:00.000Z",
  "status": "active",
  "department": "Analytics"
 },
 {
  "title": "Intern Developer",
  "company": "StartupHub BD",
  "location": "Dhaka",
  "salary": 15000,
  "experience": 0,
  "skills": ["HTML", "CSS", "JavaScript"],
  "isRemote": false,
  "applicants": 350,
  "posted": "2024-03-20T00:00:00.000Z",
  "status": "active",
  "department": "Engineering"
 }
]
```

💡 **এই ২০টি ডকুমেন্ট সফলভাবে ইনসার্ট হলে আপনি দেখতে পাবেন: `acknowledged: true`, `insertedCount: 20`**

✅ ভেরিফিকেশন

সবকিছু সঠিকভাবে ইনসার্ট হয়েছে কিনা চেক করুন:

```
db.jobs.countDocuments()  // 20 দেখাবে
db.jobs.findOne()         // প্রথম ডকুমেন্ট দেখাবে
```

Comparison Operators — $eq, $ne, $gt, $gte, $lt, $lte

🎬 ইন্ট্রো

ধরুন আপনি একটা Job Portal-এ Search করছেন — **'আমাকে ৫০ হাজারের বেশি বেতনের চাকরি দেখাও' বা 'ঢাকার বাইরের চাকরি দেখাতে চাই না'**। এই ধরনের শর্ত দিয়ে ডেটা ফিল্টার করতেই Comparison Operators ব্যবহার হয়।

৬টি অপারেটর:

- $eq — সমান
- $ne — সমান না
- $gt — বড়
- $gte — বড় বা সমান
- $lt — ছোট
- $lte — ছোট বা সমান

📌 $eq — Equal (সমান)

মানে: 'এই ফিল্ডের ভ্যালু ঠিক এটাই হতে হবে'

উদাহরণ: শুধু Dhaka-র চাকরি দেখাও

```
db.jobs.find({ location: { $eq: "Dhaka" } })
```

বা সহজে লিখতে পারেন:

```
db.jobs.find({ location: "Dhaka" })
```

> 💡 **$eq আসলে MongoDB-এর default behavior, তাই { field: value } লিখলেও একই কাজ হয়।**

📌 $ne — Not Equal (সমান না)

মানে: 'এই ফিল্ডের ভ্যালু এটা হবে না'

উদাহরণ: Dhaka ছাড়া অন্য সব জায়গার চাকরি দেখাও

```
db.jobs.find({ location: { $ne: "Dhaka" } })
```

এই কুয়েরি Chittagong, Sylhet, Rajshahi, Khulna-র চাকরি দেখাবে।

> 💡 **Real World: 'ঢাকার বাইরে যেতে রাজি আছি, তাহলে আমাকে অন্য শহরের চাকরি দেখাও।'**

📌 $gt — Greater Than (বড়)

মানে: 'এই ফিল্ডের ভ্যালু নির্দিষ্ট সংখ্যার চেয়ে বড় হতে হবে'

উদাহরণ: ৭০ হাজারের বেশি বেতনের চাকরি দেখাও

```
db.jobs.find({ salary: { $gt: 70000 } })
```

এতে Software Engineer (75k), DevOps (90k), PM (95k) ইত্যাদি দেখাবে।

📌 $gte — Greater Than or Equal (বড় বা সমান)

মানে: 'এই ফিল্ডের ভ্যালু নির্দিষ্ট সংখ্যার সমান বা বেশি'

উদাহরণ: ঠিক ৭০ হাজার বা তার বেশি বেতনের চাকরি

```
db.jobs.find({ salary: { $gte: 70000 } })
```

> 💡 **$gt এবং $gte-এর পার্থক্য: $gt মানে শুধু 'বেশি', $gte মানে 'বেশি বা সমান'।**

📌 $lt — Less Than (ছোট)

মানে: 'এই ফিল্ডের ভ্যালু নির্দিষ্ট সংখ্যার চেয়ে কম'

উদাহরণ: ৫০ হাজারের কম বেতনের এন্ট্রি-লেভেল চাকরি

```
db.jobs.find({ salary: { $lt: 50000 } })
```

এতে QA Engineer (45k), Technical Writer (40k), Intern (15k) দেখাবে।

📌 $lte — Less Than or Equal (ছোট বা সমান)

মানে: 'এই ফিল্ডের ভ্যালু নির্দিষ্ট সংখ্যার সমান বা কম'

```
db.jobs.find({ salary: { $lte: 50000 } })
```

🔗 একাধিক Comparison Operator একসাথে ব্যবহার

উদাহরণ: ৫০ হাজার থেকে ৯০ হাজারের মধ্যে বেতনের চাকরি

```
db.jobs.find({
  salary: { $gte: 50000, $lte: 90000 }
})
```

💡 **একই ফিল্ডে দুটো শর্ত দেওয়া যায়! এটা AND হিসেবে কাজ করে।**

উদাহরণ: ৩ বছরের বেশি অভিজ্ঞতা এবং বেতন ৬০ হাজারের বেশি

```
db.jobs.find({
  experience: { $gt: 3 },
  salary: { $gt: 60000 }
})
```

Logical Operators — $and, $or, $not, $nor

বাস্তব উদাহরণ: JobPortal-এ আপনি বলছেন — **'Dhaka-তে চাকরি চাই এবং বেতন ৬০ হাজারের বেশি হতে হবে'** — এটা $and। আবার বলছেন — **'React জানি অথবা Vue জানি, যেকোনোটা হলেই চলবে'** — এটা $or।

📌 $and — সব শর্তই সত্য হতে হবে

উদাহরণ: Dhaka-তে এবং বেতন ৭০ হাজারের বেশি

```
db.jobs.find({
  $and: [
    { location: "Dhaka" },
    { salary: { $gt: 70000 } }
  ]
})
```

সংক্ষেপে লেখার উপায় (implicit $and):

```
db.jobs.find({ location: "Dhaka", salary: { $gt: 70000 } })
```

> 💡 **একই ফিল্ডে দুটো শর্ত দিতে হলে $and লিখতেই হবে। কিন্তু আলাদা ফিল্ডে হলে না লিখলেও চলে।**

📌 $or — যেকোনো একটা শর্ত সত্য হলেই যথেষ্ট

উদাহরণ: Dhaka অথবা Chittagong-এর চাকরি দেখাও

```
db.jobs.find({
  $or: [
    { location: "Dhaka" },
    { location: "Chittagong" }
  ]
})
```

আরেকটা উদাহরণ: বেতন ১ লাখের বেশি অথবা Remote চাকরি

```
db.jobs.find({
  $or: [
    { salary: { $gt: 100000 } },
    { isRemote: true }
  ]
})
```

📌 $not — শর্তটা সত্য না হলে দেখাও

উদাহরণ: ৫০ হাজারের বেশি বেতনের চাকরি নয় — মানে ৫০ হাজার বা কম

```
db.jobs.find({
  salary: { $not: { $gt: 50000 } }
})
```

💡 **$not সবসময় একটা অপারেটর এক্সপ্রেশনের ভেতরে ব্যবহার হয়, সরাসরি ভ্যালুর সাথে না।**

📌 $nor — সব শর্তগুলো মিথ্যা হলে দেখাও

উদাহরণ: Dhaka-তেও না, Remote-ও না — মানে অন্য শহরে অফিসে যেতে হবে

```
db.jobs.find({
  $nor: [
    { location: "Dhaka" },
    { isRemote: true }
  ]
})
```

এই কুয়েরিতে শুধু Chittagong, Sylhet, Rajshahi, Khulna-র non-remote চাকরি দেখাবে।

Element & Type Operators — $exists, $type

এই অপারেটর দুটো একটু আলাদা ধরনের — এগুলো ভ্যালু তুলনা করে না, বরং চেক করে যে একটা ফিল্ড আসলেই আছে কিনা বা ডেটার ধরন কী।

বাস্তব উদাহরণ: **আমাদের Job Collection-এ সব চাকরিতে salary ফিল্ড আছে তো? নাকি কিছু চাকরিতে বেতন উল্লেখ করা হয়নি?** এটা চেক করতেই $exists ব্যবহার হয়।

📌 $exists — ফিল্ড আছে কিনা চেক করুন

মানে: **'এই ফিল্ড ডকুমেন্টে আছে কিনা'**

উদাহরণ: যেসব জব-এ salary ফিল্ড আছে

```
db.jobs.find({ salary: { $exists: true } })
```

যেসব জব-এ salary ফিল্ড নেই:

```
db.jobs.find({ salary: { $exists: false } })
```

> 💡 **আমাদের ফেক ডেটায় সব ডকুমেন্টে salary আছে, তাই false দিলে কিছু দেখাবে না।**

প্র্যাকটিসের জন্য একটা ডকুমেন্ট ইনসার্ট করুন salary ছাড়া:

```
db.jobs.insertOne({
  title: "Unpaid Intern",
  company: "StartupXYZ",
```

```
  location: "Dhaka",
  experience: 0
})
```

এখন আবার কুয়েরি করুন:

```
db.jobs.find({ salary: { $exists: false } })
```

এবার শুধু এই Unpaid Intern-টা দেখাবে।

📌 $type — ডেটার ধরন চেক করুন

মানে: **'এই ফিল্ডের ডেটা কোন ধরনের (string, number, boolean, array, etc.)'**

উদাহরণ: salary ফিল্ড যেগুলোতে number আছে

```
db.jobs.find({ salary: { $type: "number" } })
```

Common BSON Types:

- double (1) — দশমিক সংখ্যা
- string (2) — টেক্সট
- object (3) — embedded document
- array (4) — অ্যারে
- boolean (8) — true/false
- date (9) — তারিখ
- null (10) — null
- int (16) — পূর্ণসংখ্যা

উদাহরণ: skills ফিল্ড যেগুলোতে array আছে

```
db.jobs.find({ skills: { $type: "array" } })
```

উদাহরণ: posted ফিল্ড যেগুলোতে date আছে

```
db.jobs.find({ posted: { $type: "date" } })
```

💡 **এই অপারেটর দুটো বিশেষ কাজে লাগে — ডেটা ভ্যালিডেশন এবং মিশ্র ডেটা ক্লিনআপে।**

এই দুটো অপারেটর ডেটা কোয়ালিটি চেক করার সময় অনেক কাজে আসে।

Array Query Operators — $in, $nin, $all, $size, $elemMatch

আমাদের Job Data-তে skills একটা array ফিল্ড। একজন ক্যান্ডিডেট যখন বলে 'আমি Python জানি, আমার জন্য কোন কোন চাকরি আছে?' — এই ধরনের অ্যারে-ভিত্তিক সার্চের জন্যই এই অপারেটরগুলো।

📌 $in — অ্যারের যেকোনো একটা ভ্যালু ম্যাচ হলে

মানে: 'এই লিস্টের যেকোনো একটা ভ্যালু ফিল্ডে থাকলেই দেখাও'

উদাহরণ: Python অথবা JavaScript জানা দরকার এমন চাকরি

```
db.jobs.find({
  skills: { $in: ["Python", "JavaScript"] }
})
```

আরেকটা উদাহরণ: Dhaka, Chittagong, বা Sylhet-এর চাকরি

```
db.jobs.find({
```

```
  location: { $in: ["Dhaka", "Chittagong", "Sylhet"] }
})
```

💡 **$in একটা অ্যারে নেয় এবং ওই লিস্টের যেকোনো ভ্যালু ম্যাচ হলে ডকুমেন্ট রিটার্ন করে।**

📌 $nin — Not In (লিস্টের কোনোটাই না)

মানে: 'এই লিস্টের কোনো ভ্যালুই থাকবে না'

উদাহরণ: Python বা JavaScript ছাড়া অন্য স্কিলের চাকরি

```
db.jobs.find({
  skills: { $nin: ["Python", "JavaScript"] }
})
```

📌 $all — অ্যারের সব ভ্যালু থাকতে হবে

মানে: 'এই লিস্টের সবগুলো ভ্যালু ফিল্ডে থাকতে হবে'

উদাহরণ: Python এবং SQL দুটোই জানা দরকার এমন চাকরি

```
db.jobs.find({
  skills: { $all: ["Python", "SQL"] }
})
```

💡 **$in মানে 'যেকোনো একটা', $all মানে 'সবগুলোই' — পার্থক্যটা মনে রাখুন!**

📌 $size — অ্যারের সাইজ চেক করুন

মানে: 'অ্যারেতে ঠিক কতটা আইটেম আছে সেটা চেক করো'

উদাহরণ: যেসব চাকরিতে ঠিক ৩টা স্কিল লাগবে

```
db.jobs.find({ skills: { $size: 3 } })
```

> 💡 **$size সব ডকুমেন্ট ম্যাচ করবে যেখানে skills অ্যারেতে exactly ৩টা এলিমেন্ট আছে। Greater/Less than সাপোর্ট করে না, তবে $where বা aggregation দিয়ে করা যায়।**

📌 $elemMatch — অ্যারের এলিমেন্টে জটিল শর্ত

মানে: 'অ্যারের কোনো একটা এলিমেন্ট এই সব শর্ত মেটালে দেখাও'

এটা বোঝার জন্য আগে একটা embedded array সহ ডেটা ইনসার্ট করি:

```
db.jobs.insertOne({
  title: "Senior Developer",
  company: "BigTech",
  interviews: [
    { round: 1, score: 85, passed: true },
    { round: 2, score: 70, passed: false }
  ]
})
```

এখন: যেসব চাকরিতে কোনো ইন্টারভিউ রাউন্ড আছে যেখানে score > 80 AND passed: true

```
db.jobs.find({
  interviews: {
    $elemMatch: {
      score: { $gt: 80 },
      passed: true
    }
  }
})
```

💡 **$elemMatch ছাড়া করলে MongoDB আলাদা আলাদাভাবে চেক করতো — score > 80 হয়তো round 1-এ, আর passed: true হয়তো round 2-এ। $elemMatch নিশ্চিত করে যে একই এলিমেন্টে দুটো শর্তই পূরণ হতে হবে।**

Update Operators — $set, $inc, $push, $pull

এতদিন আমরা শুধু ডেটা পড়েছি (find), এখন ডেটা পরিবর্তন করবো।

বাস্তব উদাহরণ: একটা Job Portal-এ বেতন পরিবর্তন হলো, নতুন স্কিল যোগ হলো, আবেদনকারী সংখ্যা বাড়লো — এই সব আপডেটের জন্য Update Operators লাগে।

📌 $set — ফিল্ডের ভ্যালু সেট করুন

মানে: 'এই ফিল্ডের ভ্যালু এটা করো'

উদাহরণ: TechCorp Bangladesh-এর Software Engineer-এর বেতন ৮০ হাজার করো

```
db.jobs.updateOne(
  { title: "Software Engineer", company: "TechCorp Bangladesh" },
  { $set: { salary: 80000 } }
)
```

একাধিক ফিল্ড একসাথে আপডেট:

```
db.jobs.updateOne(
  { title: "Software Engineer" },
  { $set: { salary: 80000, status: "hiring", updatedAt: new Date() } }
)
```

> 💡 **$set দিয়ে নতুন ফিল্ড যোগ করা যায়, আবার বিদ্যমান ফিল্ড আপডেটও করা যায়।**

📌 $inc — সংখ্যা বাড়ান বা কমান

মানে: 'এই সংখ্যার সাথে আরো এতটা যোগ করো (বা বিয়োগ করো)'

উদাহরণ: Data Analyst-এ নতুন ১০ জন আবেদন করেছে

```
db.jobs.updateOne(
  { title: "Data Analyst" },
  { $inc: { applicants: 10 } }
)
```

কমাতে হলে নেগেটিভ দিন:

```
db.jobs.updateOne(
  { title: "Data Analyst" },
  { $inc: { applicants: -5 } }
)
```

💡 **$inc কাউন্টার, স্কোর, বা যেকোনো incrementing নম্বরের জন্য পারফেক্ট।**

📌 $push — অ্যারেতে আইটেম যোগ করুন

মানে: 'অ্যারেতে এই নতুন ভ্যালু যোগ করো'

উদাহরণ: Frontend Developer-এর স্কিল লিস্টে TypeScript যোগ করো

```
db.jobs.updateOne(
  { title: "Frontend Developer" },
  { $push: { skills: "TypeScript" } }
)
```

একসাথে একাধিক আইটেম যোগ করতে $each ব্যবহার করুন:

```
db.jobs.updateOne(
  { title: "Frontend Developer" },
  { $push: { skills: { $each: ["TypeScript", "GraphQL"] } } }
)
```

📌 $pull — অ্যারে থেকে আইটেম সরান

মানে: 'অ্যারে থেকে এই ভ্যালু বাদ দাও'

উদাহরণ: UI/UX Designer-এর স্কিল থেকে Sketch বাদ দাও

```
db.jobs.updateOne(
  { title: "UI/UX Designer" },
  { $pull: { skills: "Sketch" } }
)
```

💡 **$pull ম্যাচিং সব ভ্যালু সরিয়ে দেয়। শর্ত দিয়েও সরানো যায়।**

Projection & Sorting

এখন পর্যন্ত আমরা find() করলে সব ফিল্ড দেখতাম। কিন্তু বাস্তবে আমরা সব ফিল্ড চাই না — যেমন Job Card-এ শুধু title, company, salary দেখালেই যথেষ্ট। এটাই Projection।

আর Sorting হলো ফলাফলকে বেতন বেশি থেকে কম, বা নাম অনুযায়ী সাজানো।

📌 Projection — শুধু দরকারি ফিল্ড দেখান

Include করা (1 দিয়ে):

উদাহরণ: শুধু title, company, salary দেখাও

```
db.jobs.find(
  {},
  { title: 1, company: 1, salary: 1 }
)
```

এতে _id আসবে (default)। _id বাদ দিতে চাইলে:

```
db.jobs.find(
  {},
  { title: 1, company: 1, salary: 1, _id: 0 }
)
```

> 💡 **Projection-এর দ্বিতীয় আর্গুমেন্ট: 1 মানে দেখাও, 0 মানে লুকাও।**

Exclude করা (0 দিয়ে):

উদাহরণ: সব ফিল্ড দেখাও কিন্তু _id এবং applicants বাদ দাও

```
db.jobs.find(
  {},
  { _id: 0, applicants: 0 }
```

```
)
```

💡 **CRITICAL: Include এবং Exclude একসাথে ব্যবহার করা যায় না (_id ছাড়া)।**

📌 Sorting — ফলাফল সাজান

sort() মেথড ব্যবহার করুন: 1 মানে ascending (ছোট থেকে বড়), -1 মানে descending (বড় থেকে ছোট)

একটা ফিল্ড দিয়ে সর্ট:

বেতন বেশি থেকে কম অর্ডারে দেখাও:

```
db.jobs.find({}, { title: 1, salary: 1, _id: 0 }).sort({ salary: -1 })
```

একাধিক ফিল্ড দিয়ে সর্ট:

প্রথমে location A-Z, তারপর salary বেশি থেকে কম:

```
db.jobs.find().sort({ location: 1, salary: -1 })
```

📌 limit() এবং skip() — পেজিনেশন

প্রথম ৫টা চাকরি দেখাও:

```
db.jobs.find().limit(5)
```

পেজিনেশন — Page 2 দেখাও (প্রতি পেজে ৫টা):

```
db.jobs.find().skip(5).limit(5)
```

সবকিছু একসাথে — বেতন অনুযায়ী সর্ট করে Page 2:

```
db.jobs.find(
  { status: 'active' },
  { title: 1, salary: 1, company: 1, _id: 0 }
).sort({ salary: -1 }).skip(5).limit(5)
```

💡 **Real World App-এ এভাবেই পেজিনেশন করা হয়!**

Aggregation Pipeline — $match, $group, $project

Aggregation Pipeline!

সাধারণ find() দিয়ে ডেটা পড়া যায়, কিন্তু বিশ্লেষণ করা যায় না। যেমন 'প্রতিটা শহরে গড় বেতন কত?' বা 'কোন ডিপার্টমেন্টে সবচেয়ে বেশি চাকরি আছে?' — এই ধরনের analytics-এর জন্য Aggregation লাগে।

Pipeline মানে হলো এক ধাপের আউটপুট পরের ধাপের ইনপুট হয়। ঠিক কারখানার assembly line-এর মতো!

📌 $match — ডেটা ফিল্টার করুন (find-এর মতো)

উদাহরণ: শুধু active চাকরিগুলো নিন

```
db.jobs.aggregate([
  { $match: { status: "active" } }
])
```

> 💡 **$match সাধারণত pipeline-এর শুরুতে দেওয়া হয় — এটা ডেটা কমিয়ে দেয় তাই পরের ধাপগুলো দ্রুত হয়।**

📌 $group — ডেটা গ্রুপ করুন

মানে: 'একটা ফিল্ডের ভ্যালু অনুযায়ী ডেটা ভাগ করো এবং calculation করো'

উদাহরণ: প্রতিটা শহরে কতটা চাকরি আছে?

```
db.jobs.aggregate([
  { $group: {
```

```
  _id: "$location",
  totalJobs: { $sum: 1 }
 }}
])
```

প্রতিটা শহরে গড় বেতন কত?

```
db.jobs.aggregate([
 { $group: {
  _id: "$location",
  avgSalary: { $avg: "$salary" },
  maxSalary: { $max: "$salary" },
  minSalary: { $min: "$salary" },
  totalJobs: { $sum: 1 }
 }}
])
```

Accumulator Operators:

- $sum — যোগফল (বা কাউন্টের জন্য $sum: 1)
- $avg — গড়
- $max — সর্বোচ্চ
- $min — সর্বনিম্ন
- $push — গ্রুপের সব ভ্যালু অ্যারেতে রাখো

📌 $project — আউটপুট ফিল্ড কাস্টমাইজ করুন

মানে: 'pipeline-এর এই স্টেজে কোন ফিল্ড রাখবো আর কোনটা বাদ দেবো এবং নতুন ফিল্ড তৈরি করবো'

উদাহরণ: শুধু title এবং company দেখাও, _id বাদ দাও

```
db.jobs.aggregate([
 { $project: {
   title: 1,
   company: 1,
   _id: 0
 }}
])
```

নতুন calculated ফিল্ড তৈরি করুন:

```
db.jobs.aggregate([
 { $project: {
   title: 1,
   salary: 1,
   monthlySalary: "$salary",
   annualSalary: { $multiply: ["$salary", 12] }
 }}
])
```

🔗 Pipeline একসাথে — Match, Group, Project

উদাহরণ: শুধু active চাকরি নিয়ে, department অনুযায়ী গড় বেতন হিসাব করো, এবং সর্টেড আউটপুট দাও

```
db.jobs.aggregate([
 { $match: { status: "active" } },
 { $group: {
   _id: "$department",
   avgSalary: { $avg: "$salary" },
   jobCount: { $sum: 1 }
 }},
```

```
  { $project: {
    department: "$_id",
    avgSalary: { $round: ["$avgSalary", 0] },
    jobCount: 1,
    _id: 0
  }},
  { $sort: { avgSalary: -1 } }
])
```

💡 **এই একটা কুয়েরিতেই Analytics Dashboard-এর জন্য সম্পূর্ণ ডেটা পাওয়া যাচ্ছে!**

এটা MongoDB-এর সবচেয়ে শক্তিশালী ফিচার।

Advanced Aggregation — $lookup, $unwind, $facet

Aggregation-এর তিনটা advanced অপারেটর যেগুলো প্রফেশনাল অ্যাপ্লিকেশনে খুব বেশি ব্যবহার হয়।

বাস্তব উদাহরণ: Job Portal-এ যদি আলাদা আলাদা collection থাকে — একটাতে jobs, আরেকটাতে companies-এর বিস্তারিত তথ্য — তাহলে দুটো collection join করে ডেটা আনতে $lookup লাগে। এটা SQL-এর JOIN-এর মতো।

🛠️ Setup: companies Collection তৈরি করুন

প্রথমে আলাদা একটা companies collection বানাই:

```
db.companies.insertMany([
 { name: "TechCorp Bangladesh", founded: 2015, employees: 500, industry: "Software" },
 { name: "Digital Solutions", founded: 2018, employees: 120, industry: "Web" },
 { name: "AIVentures", founded: 2020, employees: 80, industry: "AI/ML" },
 { name: "CloudBase Ltd", founded: 2017, employees: 200, industry: "Cloud" },
 { name: "CreativeMinds", founded: 2019, employees: 50, industry: "Design" }
]);
```

📌 $lookup — দুটো Collection Join করুন

মানে: 'এই collection-এর ডেটার সাথে অন্য collection-এর ডেটা যুক্ত করো'

```
db.jobs.aggregate([
 {
  $lookup: {
   from: "companies",
   localField: "company",
   foreignField: "name",
   as: "companyDetails"
  }
 },
 {
  $project: {
   title: 1,
   salary: 1,
   companyDetails: 1,
   _id: 0
  }
 },
 { $limit: 5 }
])
```

💡 **$lookup-এর আউটপুটে companyDetails একটা অ্যারে হিসেবে আসে। এটা flat করতে $unwind ব্যবহার করুন।**

📌 $unwind — অ্যারে Flat করুন

মানে: 'অ্যারের প্রতিটা এলিমেন্টের জন্য আলাদা ডকুমেন্ট তৈরি করো'

```
db.jobs.aggregate([
 {
```

```
  $lookup: {
   from: "companies",
   localField: "company",
   foreignField: "name",
   as: "companyDetails"
  }
 },
 { $unwind: "$companyDetails" },
 {
  $project: {
   title: 1,
   salary: 1,
   "companyDetails.industry": 1,
   "companyDetails.employees": 1,
   _id: 0
  }
 }
])
```

📌 $facet — একাধিক Pipeline একসাথে চালান

মানে: 'একই ডেটার উপর একসাথে একাধিক ভিন্ন analysis করো'

উদাহরণ: একটাই কুয়েরিতে — বেতন রেঞ্জ ব্রেকডাউন, লোকেশন ব্রেকডাউন, এবং Department ব্রেকডাউন

```
db.jobs.aggregate([
 { $match: { status: "active" } },
 {
  $facet: {
```

```
    salaryBuckets: [
     { $bucket: {
       groupBy: "$salary",
       boundaries: [0, 30000, 60000, 90000, 150000],
       default: "Other",
       output: { count: { $sum: 1 }, avgSalary: { $avg: "$salary" } }
     }}
    ],
    byLocation: [
     { $group: { _id: "$location", total: { $sum: 1 } } },
     { $sort: { total: -1 } }
    ],
    byDepartment: [
     { $group: { _id: "$department", avgSalary: { $avg: "$salary" } } },
     { $sort: { avgSalary: -1 } },
     { $limit: 5 }
    ]
   }
  }
])
```

💡 **$facet দিয়ে Dashboard-এর সব ডেটা একটাই API Call-এ পাওয়া যায় — এটা Performance-এর জন্য দারুণ!**

এই তিনটা অপারেটর জানলে আপনি প্রফেশনাল-লেভেলের ডেটা অ্যানালিটিক্স করতে পারবেন। পরের এবং শেষ ভিডিওতে আমরা পুরো মডিউলের Summary করবো।

Module Summary — সব একসাথে

📋 কী কী শিখলাম — Quick Recap

Setup

MongoDB ইনস্টল, mongosh ব্যবহার, ২০টি fake job data insert।

Comparison Operators

$eq (সমান), $ne (সমান না), $gt (বড়), $gte (বড় বা সমান), $lt (ছোট), $lte (ছোট বা সমান)

Logical Operators

$and (দুটোই সত্য), $or (একটা সত্য হলেই হয়), $not (বিপরীত), $nor (সবই মিথ্যা)

Element & Type Operators

$exists (ফিল্ড আছে কিনা), $type (ডেটার ধরন চেক)

Array Operators

$in (যেকোনো একটা ম্যাচ), $nin (কোনোটাই না), $all (সবগুলো ম্যাচ), $size (অ্যারের সাইজ), $elemMatch (এলিমেন্টে জটিল শর্ত)

Update Operators

$set (ভ্যালু সেট), $inc (বাড়ানো/কমানো), $push (যোগ), $pull (বাদ), $addToSet (duplicate ছাড়া যোগ)

Projection & Sorting

Include/Exclude, sort(), limit(), skip() — পেজিনেশন

Aggregation Pipeline

$match (ফিল্টার), $group (গ্রুপ), $project (আউটপুট কাস্টমাইজ)

Advanced Aggregation

$lookup (join), $unwind (flat করা), $facet (একসাথে multiple analysis)

🚀 Capstone Query — সব একসাথে

এখন একটা জটিল Real-World কুয়েরি করবো। Scenario: 'আমাদের HR Team-এর জন্য একটা report চাই — Dhaka বা Remote-এ active চাকরির মধ্যে, ৩+ বছর experience দরকার এমন, department অনুযায়ী গড় বেতন, মোট jobs, এবং সবচেয়ে বেশি applicant পাওয়া top job — সর্টেড।'

```
db.jobs.aggregate([
 // Step 1: ফিল্টার — Dhaka বা Remote, active, experience >= 3
 { $match: {
  $or: [{ location: "Dhaka" }, { isRemote: true }],
  status: "active",
  experience: { $gte: 3 }
 }},

 // Step 2: Department অনুযায়ী group করো
 { $group: {
  _id: "$department",
  avgSalary: { $avg: "$salary" },
  totalJobs: { $sum: 1 },
  maxApplicants: { $max: "$applicants" },
  topJob: { $first: "$title" }
 }},
```

```
  // Step 3: আউটপুট সাজাও
  { $project: {
    department: "$_id",
    avgSalary: { $round: ["$avgSalary", 0] },
    totalJobs: 1,
    maxApplicants: 1,
    topJob: 1,
    _id: 0
  }},

  // Step 4: গড় বেতন অনুযায়ী sort
  { $sort: { avgSalary: -1 } }
])
```

📚 MongoDB অপারেটর চিটশিট

Comparison

$eq | $ne | $gt | $gte | $lt | $lte

Logical

$and | $or | $not | $nor

Element

$exists | $type

Array

$in | $nin | $all | $size | $elemMatch

Update

$set | $inc | $push | $pull | $addToSet

Aggregation

$match | $group | $project | $sort | $limit | $skip | $lookup | $unwind | $facet

---

💡 পরবর্তী পদক্ষেপ

MongoDB শেখার পর আপনি এগুলো শিখতে পারেন:

- ✅ MongoDB Indexes — কুয়েরি দ্রুত করুন
- ✅ Mongoose ODM — Node.js-এর সাথে MongoDB ব্যবহার
- ✅ MongoDB Atlas — Cloud Database সেটআপ
- ✅ Transactions — একাধিক অপারেশন একসাথে
- ✅ Schema Validation — ডেটার নিয়ম নির্ধারণ